

实验报告：修复一个可复现的线性回归训练循环

项目	内容
课程名称	系统开发工具基础
实验主题	Python 与 PyTorch
题目编号	第12题
学号	25020007191
姓名	周御基
实验日期	2026-09-07

一、实验目的

1. 学会在仅有 CPU 的 Ubuntu 服务器上安装 PyTorch。
2. 掌握 PyTorch 训练循环的标准顺序： `zero_grad` 、 `backward` 、 `step` 。
3. 掌握评估模式 `model.eval()` 与 `torch.no_grad()` 的用法。

二、实验环境

- 操作系统：Ubuntu 25.04 (x86_64, 3.3GB 内存)
- 命令行工具：bash、python3.13、venv、pip、aria2
- 其他软件：PyTorch 2.9.1+cpu (cp313, manylinux_2_28)

三、实验步骤与代码

步骤1：安装 PyTorch (CPU 版)

服务器内存小、磁盘有限，直接 `pip install torch` 会指向 800MB 以上的 CUDA 构建，且官方下载站点单连接极慢（约 10KB/s）。最终方案：用 aria2 多线程并行下载官方 CPU 版 wheel，再经清华 PyPI 安装其依赖。

```
aria2c -x16 -s16 -k1M -o ~/torch_cpu.whl \
  'https://download.pytorch.org/whl/cpu/torch-2.9.1%2Bcpu-cp313-cp313-
  manylinux_2_28_x86_64.whl'
~/lab/venv/bin/pip install ~/torch_cpu.whl -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

输出结果:

```
[#....] 下载速度约 23 MiB/s · 419MB wheel 下载完成
Successfully installed torch-2.9.1+cpu
```

验证导入:

```
~/lab/venv/bin/python -c "import torch; print(torch.__version__)"
```

输出结果:

```
2.9.1+cpu
```

步骤2: 补全 train.py 的训练循环与评估段

```
mkdir -p ~/lab/q12 && cd ~/lab/q12 && vim train.py
```

补全的关键两处 (TODO) :

```
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()    # ① 清零梯度
    loss.backward()    # ② 反向传播
    opt.step()         # ③ 更新参数

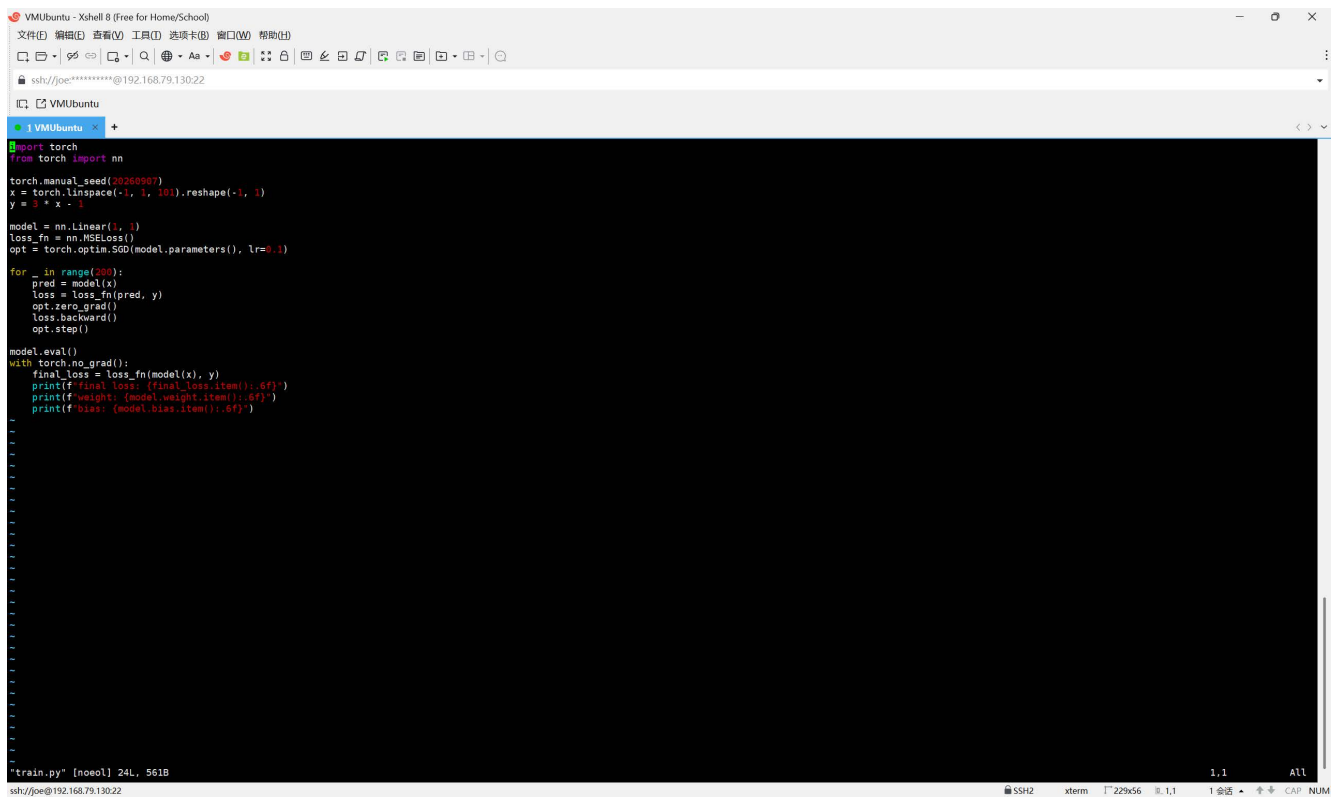
model.eval()          # 评估模式
with torch.no_grad(): # 不跟踪梯度
    final_loss = loss_fn(model(x), y)
    print(f"final loss: {final_loss.item():.6f}")
    print(f"weight: {model.weight.item():.6f}")
    print(f"bias: {model.bias.item():.6f}")
```

步骤3：运行训练脚本

```
cd ~/lab/q12 && ~/lab/venv/bin/python train.py
```

输出结果：

```
final loss: 0.000000  
weight: 2.999997  
bias: -1.000000
```



```
VMUbuntu - Xshell 8 (Free for Home/School)
文件(F) 编辑(E) 查看(V) 工具(T) 选项(O) 窗口(W) 帮助(H)
ssh://joe@192.168.79.130:22
1 VMUbuntu
import torch
from torch import nn

torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print(f'final loss: {final_loss.item():.6f}')
    print(f'weight: {model.weight.item():.6f}')
    print(f'bias: {model.bias.item():.6f}')
```

train.py [noel] 24L, 561B

四、实验结果

4.1 结果展示

- 目标函数 $y = 3x - 1$ ，训练 200 个 epoch 后：
 - 最终损失 `0.000000` (< 0.001 , 达标)；
 - 拟合的 `weight = 2.999997`、`bias = -1.000000`，与真实参数 3 和 -1 几乎一致；
- 参数由 SGD 逐步更新收敛而来，代码中没有任何直接赋值 3 / -1 的作弊行为。

4.2 结果分析

- 训练循环顺序很关键：先用 `zero_grad()` 清空上一次迭代的梯度，再 `backward()` 计算本轮梯度，最后 `step()` 更新参数；若顺序颠倒或漏调，梯度会累积导致不收敛。
- `model.eval()` 切换 BatchNorm / Dropout 行为，`torch.no_grad()` 关闭自动求图以省内存、加速推理，二者配合用于评估。
- PyTorch 的 CPU wheel 自带 OpenMP 运行库，解压后占用约 1.5GB 磁盘，故训练后删除 wheel 释放空间。

五、实验总结

通过本次实验，我掌握了以下技能：

1. 在受限环境下用 aria2 并行下载并离线安装 PyTorch CPU wheel。
2. 补全标准训练循环 `zero_grad` → `backward` → `step`，复现可收敛的线性回归。
3. 使用 `model.eval()` 与 `torch.no_grad()` 计算并打印最终损失与参数。

实验中的脚本文件：

1. `Scripts/q12/train.py`